Language specification - While

Description of the language that we will create a compiler for

This is a version of the *While language*, that is often used for education purposes.

The example on the right computes the smallest positive divisor of the number received as input.

```
# Computing the smallest positive
divisor.
program divisor
  integer a;
  integer i;
  integer divisor;
  boolean divisorfound;
begin
  read( a );
  divisorfound := false;
  i := 2;
  while not divisorfound and i < a
do
    if a mod i = 0 then
      divisorfound := true;
      divisor := i;
    endif
    i := i+1;
  done
  if divisorfound then
    write( divisorfound );
    write( divisor );
  else
    write( divisorfound );
  endif
end
```

Definition of the language:

- Source files can contain the following ASCII **characters**:
  - upper- and lower case letters of the English alphabet
  - decimal digits (0-9)

  - `():+-*<>=_;#`
  - space, tab, newline
  - within comments: any character is allowed

  The compiler has to report any violation of the rule above. The language is case-sensitive.
- **Keywords** of the language:
  `program begin end integer boolean true false div mod and or not skip if then else endif while do done read write`
- **Identifiers**: name of the variables and the name of the program have to start with a letter, and may continue with letters, digits and _. Keywords are not allowed as identifiers.
- **Types**:
  - `integer`: unsigned integer stored on four bytes; literals consists of digits and have no sign
  - `boolean`: stored on one byte; literals: `false`, `true`
- **Comments**: from # to the end of the line. Can occure at any point in the source code. They have no influence on the compilation and the object code.
- **Program structure**: Programs are composed of a signature, declarations, a body and an ending. The contents of the signature: `program` *name*, where the name is an arbitrary identifier. The signature is followed by the declarations, then the `begin` keyword starts the body. The list of declarations can be empty. The body has to contain at least one instruction. The ending is signed by the `end` keyword.
- **Variable declarations**: each variable is declared as *type name* **`;`**, each one separately (even if they have the same type).
- **expressions of type `integer`**: numeric literals, `integer`-typed variables and compound expressions composed by the + (addition), – (subtraction), `*` (multiplication), `div` (integer division), `mod` (remainder) infix operators and parentheses.
- **`boolean`-typed expressions:**: `true` and `false`, compound expressions built up by two `integer` expressions by the = (equality), < (smaller), > (greater) infix operators, and the ones composed from these by the `and` (conjunction), `or` (disjunction), = (equality) infix and the `not` (negation) prefix operators and parentheses.
- The infix operators are all left-associative and their precedence is growing in the following order:
  - `or` `and`
  - `=`
  - `< >`
  - `+ -`
  - `* div mod`
- **`skip` instruction**: proceed without modifying any of the variables.
- **Assignment**: by the **`:`**= operator. On the left hand side there is a variable, on the right hand side there is an expression that has the same type as the left hand side.
- **A `read` instruction** reads a value from the standard input into the given variable. (Implementation: call a C function that will be provided later...)
- **A `write` instruction** writes the value of the given expression (and a newline) to the standard output. (Implementation: call a C function that will be provided later...)
- **Loop**: `while` *condition* `do` *instructions* `done`

The condition is a boolean expression, the loop has to contain at least one instruction. Works as a usual while loop.

- **Branch**: `if` *condition* `then` instructions `endif`
  or
  `if` *condition* `then` instructions `else` instructions `endif`
  The condition is a boolean expression, there is at least one instructions in each branch. Works as usual.